

Pre-submission audit — `Novelx/A-GCL` @ `a40c3d4` ("August8th 2026")

Diffed against upstream `08339b7`, every `.py` compile-checked, both committed smoke-test logs read line by line.

Verdict

Don't submit yet. The repair work is good — I traced 13 of the issues I raised to a fixed line, and two of your fixes are better than the paper's own code. But it does **not** align perfectly:

- one deliberate sign change moves you away from the implementation that produced Table 2, and removes the mechanism that keeps the augmentation bounded;
- an entire code path — the memory-bank variant, which **is** the paper's headline method — has never been executed;
- two data-level assumptions are unverified and one of them is probably silently wrong;
- the README still documents the upstream data format, which your loader no longer reads.

Also: "it ran one epoch without crashing" is the one class of evidence that cannot detect any of the above. More on that at the end.

1. Verified fixed — all of it, line by line

Issue	Fix	Where
SyntaxError, unterminated call	16-value tuple unpack, call restored	<code>embedding_evaluation.py:275</code>
<code>TUEvaluator</code> not exported	re-export added	<code>datasets/__init__.py:2</code>
<code>from scipy import interp</code>	→ <code>from numpy import interp</code>	3 files
<code>running_time</code> unbound when <code>flag=False</code>	timing moved out of the <code>if</code>	<code>embedding_evaluation.py:180–186</code>
<code>test_id</code> NameError	line deleted	<code>embedding_evaluation.py:258</code>
13-vs-15 return arity	both sides now 16	<code>embedding_evaluation.py:234, 275</code>
Eval discarded edge weights	<code>self.forward(batch, x, edge_index, None, edge_weight)</code>	<code>tu_encoder.py:98</code>
Subject↔feature filename collision	exact <code>_adj.mat</code> → <code>_nf.mat</code> swap, <code>isfile</code> guard	<code>abideDataset.py:51–53</code>
Paper says 5-fold, code was 10	<code>folds=5</code>	<code>embedding_evaluation.py:239</code>
§2.1 normalisation absent	node min-max over all 3 channels; edge weights ÷ max w	<code>abideDataset.py:61–63, 73–75</code>
AUC not computable	proper <code>decision_function</code> → <code>roc_auc_score</code>	<code>embedding_evaluation.py:222–232</code>
Sensitivity/F1 were w.r.t. controls	ASD=1, NC=0	<code>abideDataset.py:92–93</code>
Memory-bank sign inverted	model now minimises <code>cr_loss</code> , view maximises — a real min-max	<code>agcl_ABIDE_queue.py:207, 233</code>

Two of these are improvements on the published code, not just repairs:

- **AUC.** The authors fed `roc_curve` hard labels, which is degenerate. You use the SVM decision function, which is the correct continuous score. Your AUC column is meaningful; theirs was not.
- **Memory-bank sign.** In upstream, both players *maximised* `cr_loss` — no adversarial structure, and the wrong direction. Your `+ cr_lambda` on the model side with `+` on the view side gives encoder-minimises / view-maximises. That is the correct reading of Eq. (2).

PyG 2.6 fix worth calling out. Job `1705057` died at `TypeError: propagate() got an unexpected keyword argument 'edge_weight'`; job `1705092` passed after you added `# propagate_type: (x: OptPairTensor, edge_weight: OptTensor)` at `wgin_conv.py:37`.

Modern PyG compiles `propagate` from that annotation. That crash is also the only positive evidence in your logs that edge weights now reach the encoder — the fix at `tu_encoder.py:98` is what exposed the missing annotation in the first place.

2. Blocker — the `reg` sign flip removes the brake

`agcl_ABIDE.py:155` and `agcl_ABIDE_queue.py:207`:

```
python

- view_loss = model.calc_loss(x, x_aug) - (args.reg_lambda * reg)
+ view_loss = model.calc_loss(x, x_aug) + (args.reg_lambda * reg)
```

`reg` is the **mean drop probability**. `(-view_loss).backward()` is ascent. So with `+`, the view learner is now *rewarded* for dropping more edges — and nothing opposes it.

That minus sign is not a bug. It is the perturbation budget. This code is a fork of AD-GCL (the author's own `.pyc` carries the path `.../gcn_project/adgcl_gcn/`), where the regulariser exists to stop the learned view from destroying the graph. Delete it and the view learner's problem has no interior optimum: both terms it maximises — `calc_loss` and `reg` — point the same way, toward an emptier graph.

Your epoch-1 log already shows it. Gates initialise at `sigmoid(0) = 0.5`, so `reg` starts at exactly 0.50. After one epoch:

```
Epoch 1, Model Loss 117.22, View Loss 155.08, Reg 0.5245771181994471
```

`Reg` is moving **up**. Under the official sign it moves down. One epoch, ~20 steps, +0.025. That single number is the fingerprint of the flip, and it is the only thing in your smoke test that tells you anything about training dynamics.

You are not wrong that the paper's §2.2.2 prose says the opposite — *"to force more edges to be dropped... we want to minimize R."* You have found a genuine paper↔code inconsistency in the original. But you have resolved it in the direction where the objective is unbounded.

The paper's prose describes a degenerate optimisation; the inherited code is the workable one.

Two further consequences:

- $\lambda_1 = 2.0$ is not a calibrated value under $\oplus$. The Fig. 6 sweep that selected $\lambda_1 = 2$ was run with the minus sign. Carrying the number over to a flipped objective transfers nothing.
- **Comparability is gone.** Every number in Table 2 came from the minus sign. If you report against 80.65% with $\oplus$, you are comparing two different objectives.

Recommendation: revert to $\ominus$ as your primary configuration. If you want to address the paper's prose — and you should, it's a real finding — run $\oplus$ as an explicitly labelled ablation and report the `Reg` trajectory for both. If `Reg` under $\oplus$ climbs monotonically past ~ 0.8 and keeps going, say so: that is the evidence that the paper's stated objective is degenerate, and it's a more interesting result than a number.

Whichever you choose, **log `Reg` every epoch and plot it.** It is the cheapest diagnostic you have and it is already being printed.

3. Blocker — the paper's actual method has never been run

Your smoke test ran `agcl_ABIDE.py`. From the log's `Namespace(...):` no `cr_lambda`, no `max_length`, no `memory_type`. That is the **no-memory-bank** variant.

`agcl_ABIDE_queue.py` — $\lambda_2 = 0.4$, bank length 256 — is the configuration behind 80.65%. It is also the only path that touches `MemoryBank_Q`, `calc_regloss`, and your `cr_loss` sign fix. **Zero lines of it have executed.**

Smoke-test it before you submit. Two specific things to watch:

- `MemoryBank_Q.__init__`: `torch.zeros((max_length, feature_dim), requires_grad=True).to(device)`. The `.to()` makes `memory` a **non-leaf** with a live `ToCopyBackward` node, and `cr_loss.backward()` traverses it every iteration. Whether torch 2.5 raises "*Trying to backward through the graph a second time*" here depends on whether that node retains buffers — I could not test it (no GPU/torch in my sandbox). If it does raise, the one-line fix is to drop `requires_grad=True`; nothing optimises this tensor and you write to it via `.data` anyway, so the flag is inert.
- **Cold-start negatives.** The bank is initialised to zeros, and `F.normalize` leaves zero rows as zeros, so every unfilled slot contributes `exp(0) = 1` to the InfoNCE denominator. At `max_length=256, batch=32` that's the first 8 steps dominated by dummy negatives. Not fatal, but mask unfilled slots or note it.

4. Verify before you submit — the PCC diagonal

`abideDataset.py:73-75`:

```
python
```

```

edge_weight_max = edge_weight.abs().max()
if edge_weight_max > 0:
    edge_weight = edge_weight / edge_weight_max

```

If `cropped_matrix` still carries the PCC diagonal of 1.0, then `edge_weight.abs().max()` is that 1.0 and **your normalisation is a no-op**. You also get 90 self-loops of weight 1, on top of the `(1 + eps) * x_r` root term `WGINConv` already adds — every node's own features counted twice.

Thirty-second check on one subject:

```

python

import scipy.io as sio, numpy as np
A = sio.loadmat('<one>_adj.mat')['cropped_matrix']
print('diag nonzero:', np.any(np.diag(A) != 0), '| max|w|:', np.abs(A).max())

```

If `max|w|` comes back as exactly `1.0000`, the diagonal is in there. Fix is one line before `sp.coo_matrix`:

```

python

np.fill_diagonal(edge_index, 0)

```

Then the max-abs normalisation operates on real correlations and the self-loop double-count disappears. Either way, state in your methods whether self-connections are retained — the paper doesn't.

5. Verify before you submit — nothing counts your subjects

`_load_class` silently `continue`s when an NF file is missing (`abideDataset.py:52-53`), and nothing anywhere logs `len(dataset)`. A naming mismatch that drops half of one class produces **no signal at all** — the run just proceeds with a smaller, more imbalanced N and slightly different accuracy.

Given you have an exclusion cascade to report, N is a number you need on the record. Add:

```

python

# in _load_class, before return
print(f"{adj_folder}: loaded {len(data_list)} / {len(adj_files)} adj files")
assert len(data_list) == len(adj_files), f"{len(adj_files)} - {len(data_list)} su

# in run(), after dataset construction
logging.info("N = %d  (ASD = %d, NC = %d)", len(dataset), int((dataset.data.y =

```

Also worth checking: the epoch-1 log shows sensitivity 0.25 / specificity 0.75 *before training*, i.e. the SVM predicts NC for most subjects. Untrained features explain some of that, but so does class imbalance. Log the counts and you'll know which.

6. Your printed losses are batch-size-scaled — this will corrupt the grid

```
python

model_loss_all += model_loss.item() * batch.num_graphs    # L181
fin_model_loss = model_loss_all / len(dataloader)         # L187 ← batches, not
```

So the printed number is $\text{batch_size} \times \text{mean per-batch loss}$. Your log's `Model Loss 117.22` at batch 32 is a true mean of **3.66**, against a chance floor of $\log(31) = 3.43$. Barely above chance, which is right for epoch 1.

If you sweep batch size, the printed loss jumps $\sim 4\times$ at 128 and $\sim 8\times$ at 256 **purely from this**, on top of the $\log(B-1)$ chance shift (3.43 / 4.84 / 5.54). Two different confounds stacked on the same number. Either fix the denominator to len(dataset) , or always report $\text{fin_model_loss} / \text{batch_size} - \log(\text{batch_size} - 1)$.

7. Evaluation protocol — unchanged, and you should say so

Still present, inherited:

- `random_state=None` on both `KFold` and `train_test_split` (`embedding_evaluation.py:257, 260`) → a **fresh split at every eval interval**. Your run is not reproducible from the seed. At minimum thread `args.seed` through.
- `best_test_epoch = np.argmax(test_curve)`, computed independently per metric (`agcl_ABIDE.py:280-289`) → the `BestTestScore` line can draw its five numbers from five different epochs.
- The encoder is self-supervised on the full dataset before any split — transductive.

For reproducing A-GCL as a baseline, keeping the original protocol is defensible. But state it, and make your nested-CV number the one you compare against. Don't let a `BestTestScore` line into the paper without a sentence explaining how the epoch was chosen.

8. Housekeeping — visible to anyone who opens the repo

- README is unchanged from upstream**. It documents `alff_value_cache`, `corr_each_sub`, and `sub_0050002_aal3_all_positive.mat`. Your loader reads `norm_matrix`, `cropped_matrix`, and `*_adj.mat` / `*_nf.mat`. This is the first file a reviewer opens and it describes a format your code cannot read.
- data is committed as a symlink** to `/users/3171356m/sharedscratch/A-GCL/data`. `.gitignore` has `data/`, which does not match a symlink named `data`. Anyone cloning gets a dangling link. `git rm --cached data`.

- **43 `.pyc` files committed.** Add `__pycache__` to `.gitignore`, `git rm -r --cached`.
- `from numpy import interp` in three files, still unused. Delete.
- `torch.load(path)` at `abideDataset.py:22` — pass `weights_only=False` explicitly. The FutureWarning in your log becomes an error on torch 2.6.
- `np.nanargmax` at `agcl_ABIDE.py:287-289` raises on an all-NaN array. Can't happen with `LinearSVC`/`SVC` (both have `decision_function`), but guard it if you add a classifier that doesn't.
- `num_dataset_features=3` hardcoded in 4 places; `--template` is still a no-op. Fine today, silent truncation if you ever add a 4th band.

9. Two modelling choices to state explicitly

Neither is wrong. Both are decisions a reviewer will ask about.

- **90 ROIs, not 116.** `cropped_matrix` says you cropped. The paper's AAL1 is 116 (cerebrum + cerebellum + vermis) and 80.65% is the 116-node number. Cropping to the 90 cerebral ROIs is a defensible choice — say that you made it and why.
- **Per-subject min-max on node features** (`abideDataset.py:61-63`), rather than dataset-wide. This is the *right* call — dataset-wide statistics would leak test-fold information into training, exactly the issue you fixed elsewhere. But note the consequence: mALFF is already normalised to whole-brain mean 1, and a second per-subject min-max removes between-subject amplitude entirely. Your node features are pure spatial pattern. One sentence in methods.

10. On the smoke test

`epochs=1, eval_interval=1` proves the process starts, loads data, completes a forward/backward pass, and writes a log. It cannot detect a sign error, an unbounded objective, a silently-halved cohort, a no-op normalisation, or an untouched code path. All five of the things above sit in that blind spot.

And the numbers themselves are a mild warning, not reassurance:

	acc	AUC
before training	0.5094	0.5063
after epoch 1	0.4833	0.4914

Test accuracy and AUC both went **down**, below chance. At one epoch that is not alarming on its own — but it means the run gave you no positive evidence about correctness either.

What would actually be evidence, in order:

1. Revert `reg` to `-`, run 100 epochs, plot `Reg` and `Model Loss / batch_size - log(B-1)`. `Reg` should settle, not run to a boundary.

2. Run the same 100 epochs with $\oplus$. Compare the Reg curves. That comparison is your paper↔code finding, with data behind it.
3. Smoke-test `agcl_ABIDE_queue.py` for 2 epochs — 2, not 1, so the memory-bank backward path runs more than once.
4. Print the diagonal check and the subject counts. Two lines, removes two unknowns.

Items 3 and 4 cost minutes. Items 1 and 2 cost one afternoon and are the difference between "I ran the official method" and "I ran something adjacent to it."